

Assignment 1

Task 1: Forecasting Across Heterogeneous Sensors

Task 2: Leaderboard Challenge

Rana Taqveem Roll number 25280030

Repository: https://github.com/rana-taqveem/msai_dl4stg

Task 1: Forecasting Across Heterogeneous Sensors

Evidence: Saved full-preset run of `Assignment1.ipynb`, data and model seeds 0, CPU. Stations 1–3 are established; Station 4 is held out from fitting. Context length is 96 and forecast horizon is 48 samples. Unless stated otherwise, errors are validation errors; RMSE uses m/s^2 and MSE uses $(\text{m/s}^2)^2$. Original numbered figures are retained.

Table 1: Outputs 1.1 and 4.1: validation error and fitting cost. Period-routed ridge leads both populations; Autoformer-inspired leads the neural models. A dash means no fitted Station-4 model. Times are CPU fitting times, not inference latency.

Model	Established		Station 4		Parameters	Fit (s)
	RMSE	MSE	RMSE	MSE		
Shared ridge	0.767	0.588	0.876	0.767	4,608	0.007
Sensor-specific ridge	0.768	0.590	—	—	13,824	0.015
Period-routed ridge	0.166	0.028	0.173	0.030	59,904	0.043
Raw Attention	0.431	0.186	0.612	0.374	368,368	4,990.703
Attention + decomposition	0.296	0.087	0.430	0.185	373,024	7,021.893
Raw delay mixer	0.225	0.051	0.333	0.111	368,370	10,483.835
Autoformer-inspired	0.199	0.040	0.240	0.057	373,026	8,129.553

1 Response 1A: Predict, measure, and revise the ordering

1.1 Recorded expectation

Raw attention is expected to come out ahead of other approaches because of the understanding that learned projections can better explain the repeating structures. This allows the model to construct weights that depend on the repeating pattern in the input context instead of a fixed matrix. Next in the ranking would be period-routed ridge, as explicit filters are crafted each period; however, these filters will come short probably because of the limitation in period estimation and discrete routing bands. Shared ridge is expected to slightly outperform the sensor-specific ridge because the station identity does not add any meaningful information that helps in understanding the main pace variation and separate fits use fewer examples.

Population	Expected ordering, best to worst
Established	Raw Attention; period-routed ridge; shared ridge; sensor-specific ridge.
Station 4	Raw Attention; period-routed ridge; shared ridge. Sensor-specific ridge unavailable.

1.2 Measurements and revised ranking

Table 2: Output 1.1: aggregate validation performance.

Model	Established		Station 4	
	RMSE	MSE	RMSE	MSE
Shared ridge	0.767	0.588	0.876	0.767
Sensor-specific ridge	0.768	0.590	—	—
Period-routed ridge	0.166	0.028	0.173	0.030
Raw Attention	0.431	0.186	0.612	0.374

Table 3: Output 1.1: validation RMSE by product and population.

Model	Established			Station 4		
	A	B	C	A	B	C
Shared ridge	0.805	0.854	0.621	0.909	0.982	0.713
Sensor-specific ridge	0.804	0.852	0.630	—	—	—
Period-routed ridge	0.181	0.164	0.152	0.184	0.182	0.150
Raw Attention	0.388	0.482	0.418	0.647	0.580	0.607

Period routed ridge reported the best performance for every product on both the established stations and station 4. On established stations it reduced RMSE by 78.4% relative to shared ridge and 61.5% relative to Raw Attention, and by 80.3% relative to shared ridge and 71.7% relative to raw attention on the station 4. Although raw attention showed better results by 30.1% on Station 4 over the shared ridge.

The revised ordering is therefore

$$\text{Period-routed ridge} < \text{Raw Attention} < \text{Shared ridge} \approx \text{Sensor-specific ridge},$$

where $<$ denotes lower error. The standard shared ridge did report a tiny edge over the station specific ridge however The 0.001 difference between the two is too small to claim a rank. Routing’s RMSE showed a slight increase of about 4.2% from established stations to Station 4, whereas the Raw Attention’s increased about 42%.

1.3 Why the models differ

Shared ridge applies one shared weight matrix on all data points for across the products:

$$\widehat{W} = \left(X^\top X/n + \lambda I \right)^{-1} X^\top Z/n, \quad \widehat{z} = x^\top \widehat{W}. \quad (1)$$

This closed form solution allow model to minimize average squared error on all varying periods. Therefore same weights must accommodate to the different periods of each product, limiting its learning ability. The sensor specific ridge learn these coefficients per station, however since the stations still observe the same pace variations therefore it does not add any additional value to the model representation to learn the critical period variations among the products. Thus it only conditions on a variable which does not help in addressing the ambiguity data has.

Period-routed ridge instead uses

$$\widehat{z} = x^\top \widehat{W}_{r(\widehat{P}(x))}, \quad (2)$$

where $\widehat{P}(x)$ is estimated from the observed context and r chooses a period band. Restricting each matrix to a narrower range of phase advances makes the linear prediction problem easier. Pace is shared across stations, so this conditioning variable transfers naturally to the held-out sensor.

Raw attention does offer more flexibility; however, it needs to learn the useful differential representations all by itself from the mixture of drift and vibrations. Its context-dependent working does show better results than the fixed shared ridge solutions; however, it is still not enough to match or outrank the handcrafted per band weights.

Response 1B: Why the forecasting rule depends on context

Figure 1 shows fixed stations with changing pace. Shared and sensor-specific ridges flatten out the later Product-A oscillations and suppress the second Product-C bump. These shape differences highlight the compromise made by the closed-form weights matrix. i.e. one fitted set of coefficients must accommodate the phase differences, resulting in averaging the oscillations. The period-routed ridge managed to follow the repeating shape more closely in these examples.

In the case of Raw Attention WQ, WK, and WV, the learned projections remain fixed; however, with different contexts, they produce different attention queries, keys, and values. Thus the resulting scores and attention weights change, giving a different value-mixing operation without refitting. Figure 2 shows this change directly. Though the ridge’s output also changes with its input, the mixing coefficients remain fixed. **The attention maps show it better utilizes the available context; however, it cannot be claimed that it managed to learn the physical period, as the expectation was laid before the execution.**

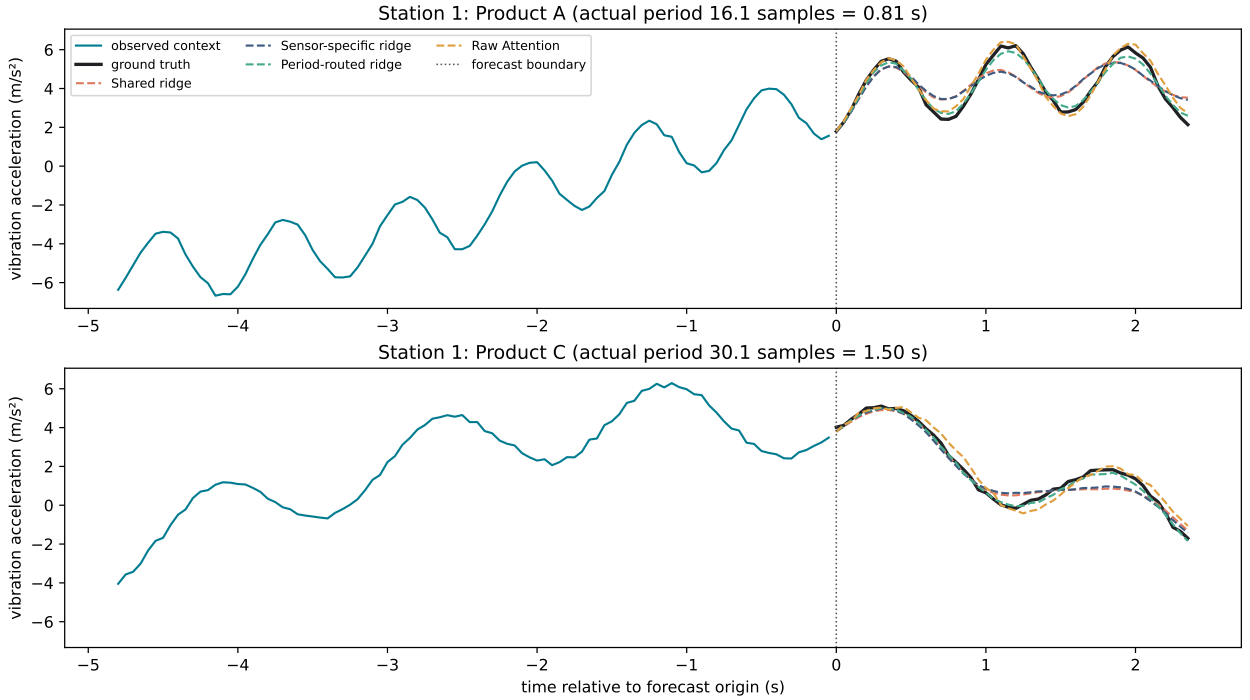


Figure 1: Output 1.2: Station 1 under Product A (period 16.135 samples) and Product C (30.093 samples). The dotted line marks the forecast origin. Unrouted ridge attenuates later oscillations; routing preserves their shape more closely. These are illustrative windows, not aggregate performance estimates.

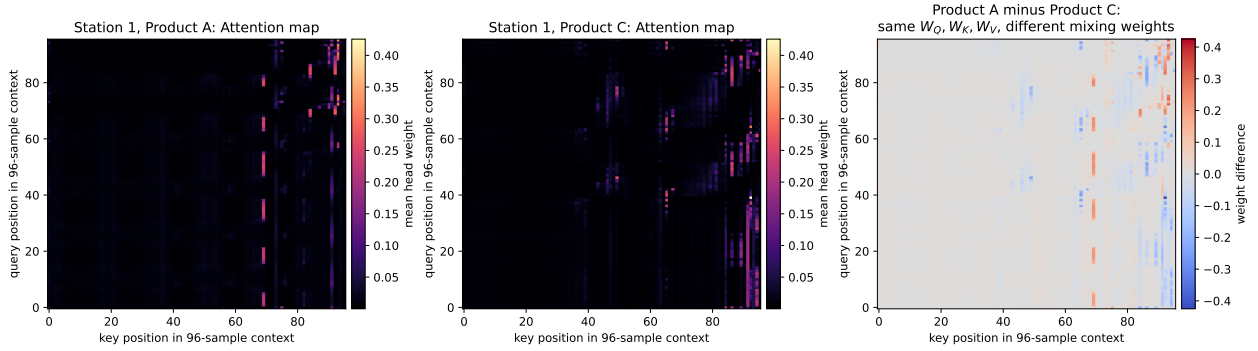


Figure 2: Output 1.3: first-layer, head-averaged Attention maps for the same contexts and their difference. Rows are queries; columns are keys. Mean absolute weight difference is 0.011 and maximum difference is 0.425. The selected keys change despite fixed learned projections; this does not establish physical-period recovery.

Response 2: Decomposition and width selection

The moving average is represented by a fixed-length 1D convolution with uniform weights. This convolution smooths out the readings, allowing us to estimate the slow-moving baseline component. Subtracting that estimate from the signal gives the underlying faster repeating vibrations, some noise, and any remaining drift.

$$T_t = \frac{1}{m} \sum_{u=-q}^q \tilde{x}_{t+u}, \quad S_t = x_t - T_t, \quad m = 2q + 1. \quad (3)$$

Why the bias fits.

Drift changes much more slowly than vibration, so averaging can retain the slow baseline while reducing the faster repetition. Results obtained with attention on the remainder improved the aggregate validation performance, supporting the moving average as a useful inductive bias here.

Selecting a window size too small may fail to cover the complete cycle and thus may end up only capturing a growing peak or a declining trough, making the average also reflect an increase or decrease. Similarly, a window too wide relative to the baseline’s timescale may smooth away meaningful changes in the baseline. The diagnostic results compare the candidate window lengths.

Selected width and observations. The selected width is **41 samples**. As shown in Figure 3 it manages to effectively separate the baseline and vibration, whereas the smaller windows tend to absorb some of the vibrations into the baseline. Once we have the baseline captured, subtracting this baseline makes the repeating vibration easier to identify. The separation remains approximate because averaging can retain some vibration in the trend, and endpoint replication affects the boundary estimates.

Table 4: Output 2.2: oracle period recovery on established-station training windows. Width 41 is selected by recovery within one sample; width 25 minimizes median error. The selection criterion favors avoiding large misses, not the greatest typical precision.

Representation	Recovery within one sample	Median error (samples)
Raw	68.67%	0.525
Width 17	98.00%	0.157
Width 25	98.92%	0.122
Width 33	99.33%	0.142
Width 41	99.85%	0.216

Synthetic data validation. Synthetic data with known periods allowed comparison of the period estimated from each remainder with the generator’s known period, using only established-station

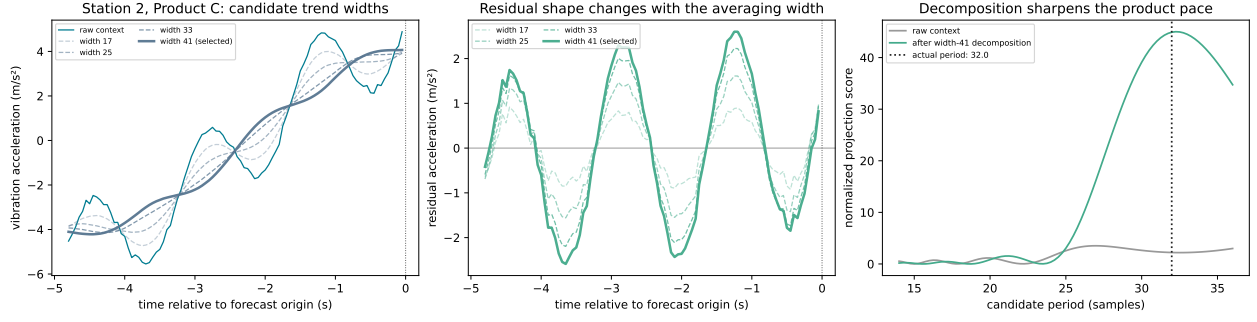


Figure 3: Output 2.1: Station 2, Product C, actual period about 32 samples. Narrow averaging absorbs more vibration into the trend; the selected width leaves a stronger repeating remainder. The right panel contrasts the weak raw period-projection response with the clearer decomposed peak.

training windows. Table 4 shows that width 41 achieved recovery within one sample for 99.85% of these windows, supporting its selection as the width that makes repetition most reliably identifiable under this criterion. Validation and Station 4 were excluded from width selection, and the known period was used only to score the diagnostic, not supplied to the forecaster.

Table 1 shows that adding decomposition improves validation forecasting on both established stations and Station 4, approximately halving MSE. This checks forecasting benefit separately from period recovery.

Failure case. An abrupt level shift violates the smooth-drift assumption: the average smears the jump and leaves an artificial transient in the remainder.

Response 3: Delay aggregation and recurrence

The aggregation implementation passes the notebook’s numerical and indexing-direction checks.

Recurrence evidence. The diagnostic in Figure 4 shows the strong interior recurrence peaks at lag 16 and 32 samples. These approximately correspond to the actual period $P \approx 16.135$ samples of the operating runs and its first two integer multiples, P and $2P$.

The faster vibrations in raw signal are dominated by the slow drift. Removing this drift via decomposition reduced the broad background correlation and made the faster repeating peaks more prominent. This supports the idea of delay mixer’s inductive bias because readings separated by period multiples are more likely to be similar and can provide useful information for forecasting.

Failure case. An isolated impact such as a sudden spike violates the delay mixer’s assumed inductive bias, as the spike has no recurring counterpart. Thus, mixing a spike with delayed values could influence unrelated values and might produce a forecast that is not a correct representation of the underlying signal.

The figure shows correlation patterns that are a direct signal diagnostic, whereas the trained mixer instead scores learned query/key features.

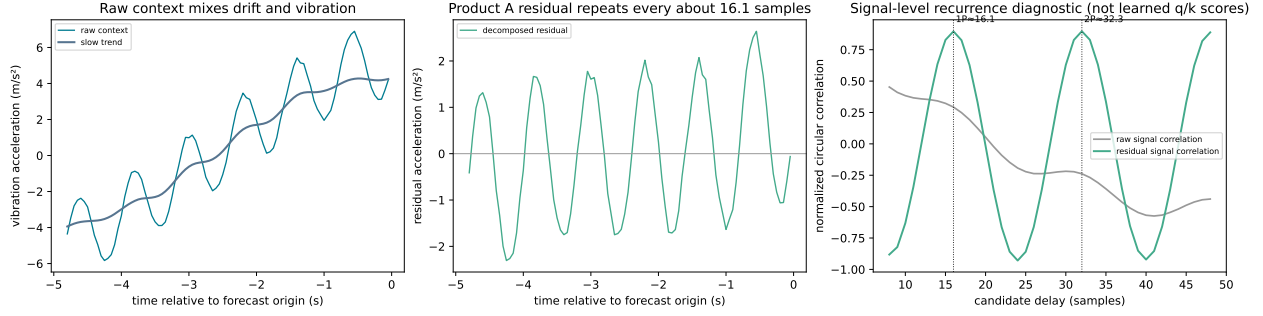


Figure 4: Output 3.2: Station 2, Product A, $P \approx 16.135$ samples. Residual correlation peaks at lag 16 (0.896) and lag 32 (0.897), near P and $2P$. The rise toward lag 48 approaches $3P \approx 48.404$, outside the band. Gray raw-signal correlation is dominated by broad drift effects. These are signal correlations, not learned model scores.

Response 4: Compare designs and deploy

(a) **Prediction.** Decomposition is expected to help most when slow drift dominates, because it separates the baseline from faster vibrations. Combining it with delay mixing should help the mixer focus on repetition.

(b) **Validation comparison.** Table 1 reports validation RMSEs, MSEs and fitting costs. Table 5 shows that both switches improve aggregate performance in both populations; their combination is best among neural models.

However, established Product A favors raw delay mixing, while Products B and C benefit from decomposition. Figure 5 shows individual gains and losses, so aggregate improvement is not universal.

Table 5: Output 4.1: relative RMSE reduction when changing one architectural switch while holding the other fixed. Positive values indicate improvement over the corresponding unswitched model; percentages use rounded reported RMSEs.

Change	Established	Station 4
Add decomposition to Attention	31.3%	29.7%
Add decomposition to delay mixing	11.6%	27.9%
Replace Attention with delays, raw input	47.8%	45.6%
Replace Attention with delays, decomposed input	32.8%	44.2%

Errors by horizon. Figure 6 shows that errors generally increase with forecast distance, especially for Raw Attention. The combined model controls later errors better than either individual upgrade; period-routed ridge remains competitive. Accumulating phase mismatch and imperfect drift extrapolation could explain this growth.

(c) **Deployment choice.** I select **period-routed ridge for both populations**: it has the lowest validation error, fewer parameters and shorter fitting time (Table 1). Its Station 4 performance supports transfer without fitting that station. Final test errors remain close to validation (Table 6), supporting this choice in the assignment’s drift-plus-repetition setting.

Table 6: Output 4.3: period-routed ridge, selected using validation before final testing. Similar validation and test errors support the selection for both populations.

Population	Validation		Final test	
	RMSE	MSE	RMSE	MSE
Established	0.166	0.028	0.161	0.026
Station 4	0.173	0.030	0.183	0.033

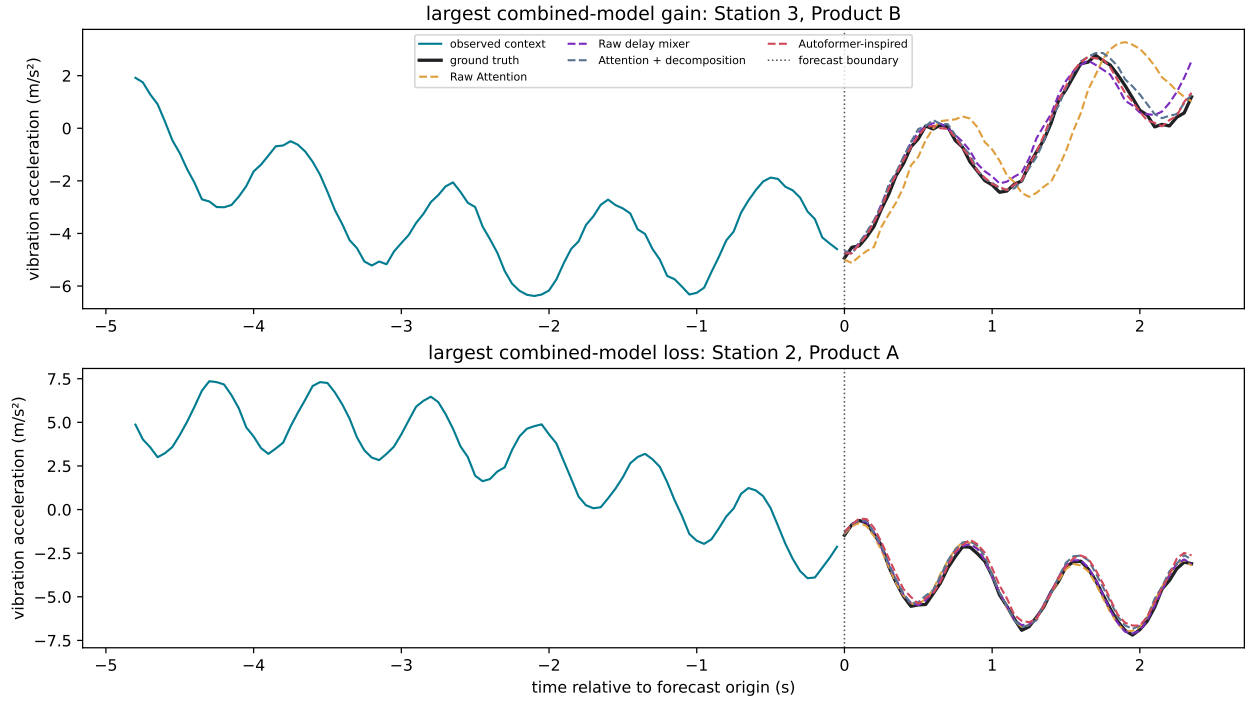


Figure 5: Output 4.1: selected gain and loss cases for the combined model. In Station 3/Product B, Raw Attention and combined-model window RMSEs are 1.325 and 0.145; in Station 2/Product A they are 0.206 and 0.375. These selected extremes show that aggregate improvement is not universal.

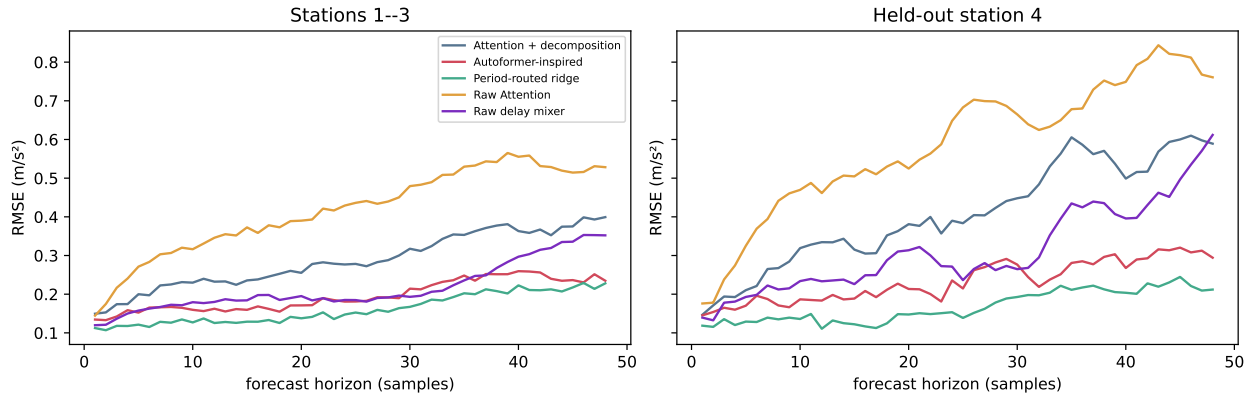


Figure 6: Output 4.2: per-horizon validation RMSE for established stations and Station 4. Errors generally rise, but are not monotonic. The combined neural model limits later error growth; period-routed ridge remains among the lowest-error curves.

Task 2: Leaderboard Challenge

Submitted model. An Autoformer (Wu et al., 2021) with known future covariates driving its trend, averaged over five seeds. Declared $P = 188,170$ ($5 \times 37,634$) and $E = 58$ ($9+20+13+6+10$ epochs run). Validation RMSE 51.07; leaderboard RMSE 80.07, MAE 48.30, sMAPE 51.22%.

Validation. All decisions were made on a chronological split: train on indices 1–39288, validate on the last 26 weeks with one 168-step forecast per day (176 blocks). Scaling statistics come from training data only. Every configuration was run with at least three seeds.

Data processing and feature engineering

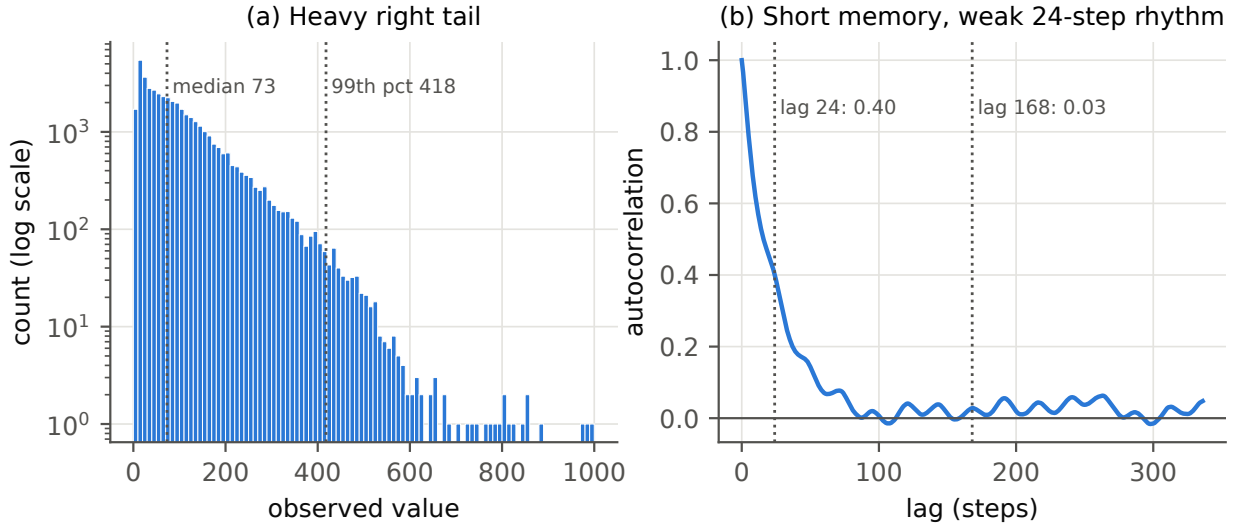


Figure T2.1: (a) The target has a long right tail. (b) Its memory fades within about two days: autocorrelation 0.40 at lag 24 and 0.03 at lag 168.

- **Global scaling, not per-window.** Over a 168-step horizon, always predicting the training mean (RMSE 88.7) beats repeating the last day (119.6) or the last value (124.1). The long-run level is the most useful information, so one mean and SD are used for the whole series instead of re-centring each window.
- **Covariate cleaning.** D, E and F are cumulative counters that are mostly zero with rare huge values, so they get $\log(1 + x)$ before scaling. G–J are one 4-state variable in one-hot form, so they stay 0/1.
- **Engineered features (18, from the external file only).** The target builds up and clears over hours, so a single row of conditions is not enough. I added trailing summaries: 24-step means (of A and of $B - A$), 3- and 24-step changes of C, recent means of D and its per-step increments, how long E, F and each G–J state have been active. All are available for the forecast week too.
- **Why covariates matter most.** A linear regression on these future features alone, with no target history, reached RMSE 66.2, already better than the standard Autoformer (68.3). The predictable signal is mostly in the known future conditions.
- **Non-negative output.** Forecasts below zero are clipped to zero.

Model architecture

Both required mechanisms are kept: a moving-average decomposition (width 25) inside every encoder and decoder layer, and Auto-Correlation, which scores time delays with an FFT and mixes the top $\lfloor \ln L \rfloor = 5$ delayed copies instead of using point-wise attention. Delays are chosen per example, as in Task 1.

Table T2.1: Design decisions and the reason for each.

Decision	Reason
Covariates: past in the encoder, future in the decoder	Future conditions are known at forecast time, so each future step receives the conditions for that step. Future target values are never given.
Trend starts from the covariates	In standard Autoformer the seasonal part is forced to average zero, so covariates entering there can add wiggles but cannot raise or lower the week. The trend starts flat at the recent average, which alone scores RMSE 96.1. A small convolution over the future covariates now sets the starting level of each future step. It starts at zero, so training begins from standard Autoformer.
Context of 336 steps	More history gives Auto-Correlation more repeats to match. It beat 168 steps in all five seeds and adds no parameters.
Small model (37,634 parameters)	A model with $3.4\times$ more parameters was no better and less stable.
All 168 steps at once	Feeding predictions back in step by step would let errors grow over a long horizon.
MSE loss, early stopping	MSE is what RMSE measures. Training stops after 5 epochs without validation gain, and the best epoch is kept.
Average of 5 seeds, no refit	Averaging cancels part of each model’s random error (single model 54.84, average 51.07). These are the exact models that were validated; a refit would be untested. The extra P and E cost only 0.03 points.

Evidence across seeds

Table T2.2: Validation RMSE, mean $\pm$ SD over seeds, and RMSE of the seed-averaged forecast. E0 rows are the required ablation of the optional file.

Configuration	Change	Seeds	RMSE	Averaged
E0_none	standard Autoformer, <i>without</i> the file	3	96.14 ± 0.35	96.01
E0_baseline	standard Autoformer, <i>with</i> the file	3	68.27 ± 2.48	65.24
E1	+ trend from covariates	3	69.16 ± 2.68	66.49
E2a	+ engineered features	3	67.74 ± 2.47	64.04
E2	+ both	5	57.88 ± 2.16	53.09
E6	E2, wider model	5	56.87 ± 3.92	51.61
E7 (submitted)	E2, context 336	5	54.84 ± 1.70	51.07

- **The optional file is essential.** It lowers RMSE in every seed (by 24.8, 29.6 and 29.3). Without it the model is worse than predicting the training mean (96.1 vs 88.7).
- **The two main changes give the largest gain together.** The covariate trend alone (E1) does not improve the mean RMSE, while the features alone (E2a) give a small reduction relative to the seed variability; together (E2) they cut RMSE by about 10. The features carry the information, and the trend pathway lets that information set the level.
- **E6 vs E7 is a tie.** E7 was chosen for its lower seed spread and smaller size.

Reference. H. Wu, J. Xu, J. Wang and M. Long, “Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting,” *NeurIPS*, 2021.

Generative-AI assistance record

AI tools were used as a guide throughout this assignment: to understand the problem statement and scope of work, to help generate code (including the Autoformer implementation), to produce visuals, and to gather and summarise results. I used them to learn and check my understanding, not as an implementer in place of my own work.